Group No.:5

Group Members: Ayushi Ray	**PRN No.** : 24070521145	**Sem/Sec:** 4th C
Shruti Bawankar	24070521148	4th C
Radhika Chaudhari	24070521150	4th C

ENTITY EXTRACTION FOR DATES & COURSE

Problem Statement

In many educational institutions, students often face difficulty in quickly finding accurate information about their exam schedules. Exam dates are usually shared through notice boards, PDFs, or institutional websites, which may require students to manually search through large amounts of information. This process can be time-consuming and inconvenient, especially when students only need a specific detail such as the exam date for a particular course and semester.

Additionally, students may phrase their queries in different ways using natural language, making it challenging for traditional search systems to understand their requests effectively. Therefore, there is a need for an automated system that can interpret student queries and provide precise exam schedule information instantly.

The problem addressed in this project is to design and implement a rule-based natural language query system that can understand user inputs, extract relevant entities such as Course and Semester, and retrieve the corresponding exam date from a predefined database. The system should be able to process variations in user input, provide accurate responses, and guide the user when incomplete information is given.

The goal is to create a lightweight, efficient, and user-friendly chatbot-like interface that simplifies access to academic schedule information without requiring manual searching through official documents.

2. Objectives

The primary objectives of this project are:

1. **To implement a Rule-Based Entity Recognition system** using Regular Expressions (Regex) to identify key entities such as Semester and Course from user queries.
2. **To automate the retrieval of exam schedule information** by matching extracted entities with a predefined database.
3. **To provide an interactive conversational interface** that allows students to easily check exam dates using natural language.

4. **To demonstrate the use of basic NLP techniques** such as pattern matching, text parsing, and structured data retrieval.
5. **To design a lightweight system** that can run efficiently without requiring heavy machine learning models or large datasets.

3. Methodology

The system follows a **modular design approach**, where each component is responsible for a specific task. This improves readability, maintainability, and scalability of the system.

The main stages of the methodology are:

3.1 Input Acquisition

The system first captures the **raw text input** provided by the user.
This input can be in different forms such as:

- "CS semester 5 exam date"
- "When is IT sem 3 exam?"
- "ME sem 2 schedule"

The input is accepted as a **string** and passed to the entity extraction module.

3.2 Entity Extraction

The entity extraction module identifies important keywords using **Regular Expressions**.

The two main entities extracted are:

- **Course Code** (CS, IT, ME, etc.)
- **Semester Number**

Regex functions such as re.search() are used to scan the query and match predefined patterns.

Example patterns include:

- (?:sem|semester)\s*(\d+) → identifies semester numbers.
- \b(CS|IT|ME|CE|EE|EC)\b → identifies course codes.

The extracted entities are stored in a dictionary structure.

Example output:

```
{
  "course": "CS",
  "semester": "5"
}
```

3.3 Data Mapping

Once entities are extracted, the system matches them against a **predefined exam schedule database**.

The database is implemented using a **dictionary data structure** where the key is a tuple of (Course, Semester) and the value is the corresponding exam date.

Example:

```
exam_schedule = {
  ("CS", "5"): "12 May 2026",
  ("IT", "3"): "15 May 2026",
  ("ME", "2"): "10 May 2026"
}
```

Using this approach allows **constant time lookup complexity (O(1))**, making the system efficient even when the database grows.

3.4 Response Synthesis

After the lookup operation, the system generates a **human-readable response**.

Three possible scenarios exist:

1. **All entities found and schedule available**

Example response:

"The exam date for CS Semester 5 is 12 May 2026."

2. **Entities found but schedule unavailable**

Example response:

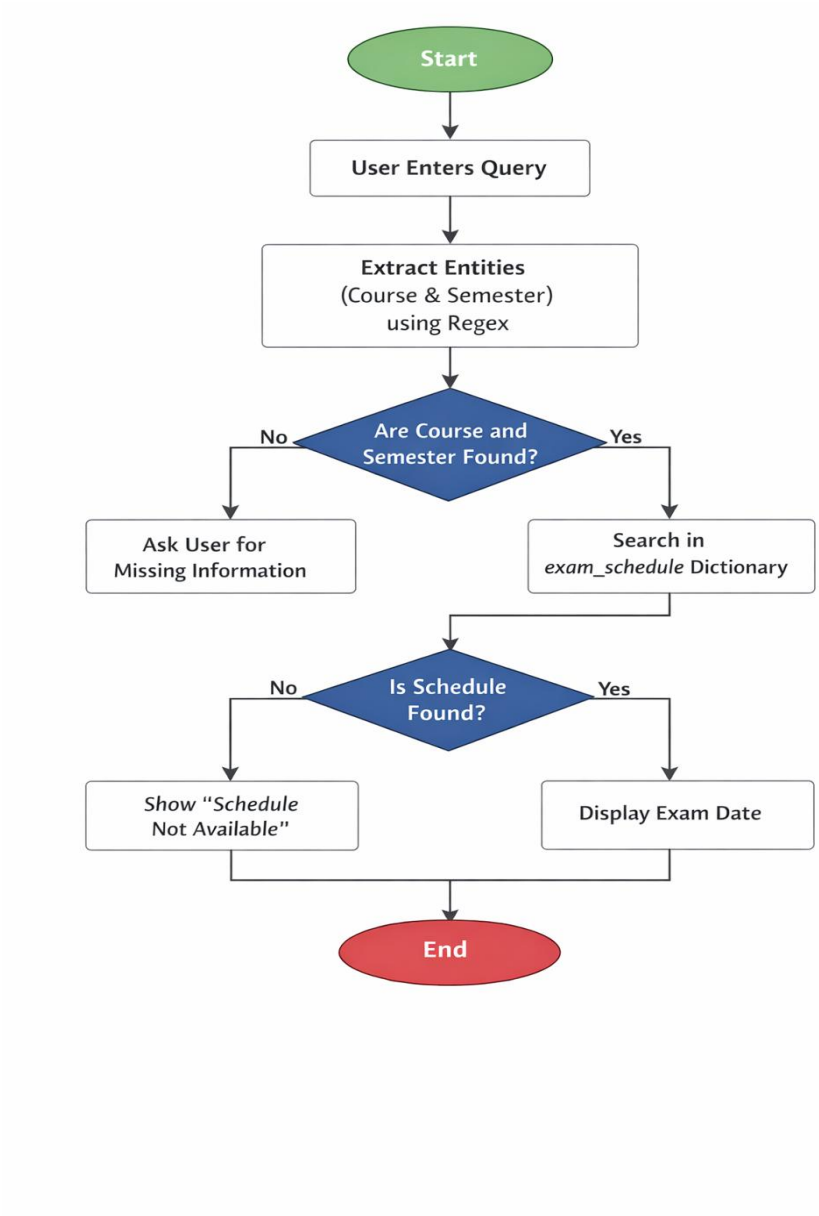
"The exam schedule for this course and semester is not available."

3. **Incomplete information**

Example response:

"Please provide both the course and semester."

4. FLOWCHART



5. TECHNOLOGIES USED

Python 3.13: The core programming language used for script execution and logic implementation.

Regular Expressions (re module): A built-in Python library used to define and match complex text patterns for extracting specific entities.

Jupyter Notebook: The interactive development environment used to write, test, and document the code in a cell-based format.

Dictionary-based Mapping: A Python data structure used as a lightweight "database" to store and retrieve exam dates based on unique entity pairs.

Natural Language Processing (Rule-Based): A technique for teaching machines to understand human language using fixed patterns rather than statistical models.

Deterministic Logic Flow: A programming methodology where the output is determined solely by specific input rules and predefined conditions.

6. System Design

The system is designed using **three main layers**, where each layer performs a specific task.

6.1 Extraction Layer

This layer is responsible for **reading the user's query and extracting important information** from it. It uses regular expressions to identify keywords in the input.

Main tasks of this layer are:

- Finding the **course name** from the query
- Finding the **semester number**
- Converting the text into a standard format so that inputs like **"cs", "CS", or "Cs"** are treated the same

This helps the system understand the user query properly.

6.2 Logic Layer

The logic layer controls **how the system responds** after the entities are extracted.

The main function used here is **generate_response()**.

This layer checks:

- Whether both **course and semester** are provided by the user
- Whether the exam schedule exists in the database
- What message should be returned to the user

Based on these conditions, the chatbot generates the appropriate response.

6.3 Storage Layer

The storage layer contains the **exam schedule database**.

It is implemented using a **dictionary** in Python where the key contains the course and semester, and the value contains the exam date.

Example:

exam_schedule = {("Course","Semester"): "Date"}

This structure allows the system to **quickly find the exam date** for a particular course and semester.

7. Model Working Mechanism

The system works using **deterministic pattern matching**, which means it follows predefined rules to identify information from the user query. It does not use machine learning or training data.

Instead, it uses **Regular Expressions (Regex)** to detect patterns in the text.

Semester Pattern

(?:sem|semester)\s*(\d+)

This pattern is used to find the **semester number** in the query.

It works by:

- Looking for the word **"sem" or "semester"**
- Allowing some **spaces after the word**
- Then capturing the **number of the semester**

Example matches:

- **"sem 5"**
- **"semester 3"**

Course Pattern

\b(CS|IT|ME|CE|EE|EC)\b

This pattern is used to identify the **course code**.

Here:

- \b ensures that only complete words are matched
- It prevents matching parts of other words
- It only accepts valid department codes like **CS, IT, ME, CE, EE, EC**

7. Model Working Mechanism

The model operates using **Deterministic Pattern Matching**.

Unlike machine learning models that require training data, this system follows **predefined rules**.

Semester Pattern

(?:sem|semester)\s*(\d+)

Explanation:

- sem or semester
- followed by optional spaces
- followed by digits

Example matches:

- "sem 5"
- "semester 3"

Course Pattern

\b(CS|IT|ME|CE|EE|EC)\b

Explanation:

- \b ensures word boundaries
- prevents partial matches
- captures valid department codes only

Lookup Mechanism

After extracting the entities:

1. A tuple key is created.

Example:

("CS","5")

2. The key is searched in the exam_schedule dictionary.
3. If found → the exam date is returned.

8. Functional Components

8.1 extract_entities(query)

This function parses the user query and extracts the relevant entities.

Responsibilities:

- Apply regex patterns
- Capture course and semester
- Return results as a dictionary

8.2 generate_response(entities)

This function generates the chatbot's response.

Responsibilities:

- Validate entity completeness
- Perform database lookup
- Construct readable responses

8.3 chatbot()

This function runs the **main conversational loop**.

Features:

- Continuously accepts user queries
- Processes them
- Returns responses

- Stops when user types **"exit"**

9. Features

Flexible Input Handling

The system accepts multiple input variations:

- "Sem 5"
- "Semester 5"
- "CS sem 5 exam"

Case Insensitive Recognition

The system treats:

- "cs"
- "CS"
- "Cs"

as the same entity.

Intelligent User Guidance

If the user provides incomplete information, the chatbot prompts for missing details.

Example:

User: "Exam date for CS"

Bot:

"Please specify the semester number."

10. Advantages

Fast Response Time

The system generates results instantly because it relies on simple rule-based logic.

Lightweight Implementation

No external machine learning libraries or datasets are required.

High Precision in Closed Domain

Since the system operates within a predefined vocabulary, it produces accurate results for expected queries.

11. Limitations

Rigid Parsing

The system only recognizes predefined patterns.

Example failure cases:

- "Computer Science" instead of "CS"
- slang or spelling variations

Static Database

The exam schedule is hardcoded, meaning updates require modifying the source code.

Stateless Interaction

The system does not remember previous user inputs in a conversation.

Example:

User: "CS exam date?"
Bot: "Which semester?"

User: "5"

The system cannot connect the second query with the first.

12. Future Enhancements

Database Integration

The exam schedule can be stored in a **SQL database**, allowing dynamic updates.

Fuzzy Matching

Libraries like **TheFuzz** can be integrated to handle spelling mistakes and variations.

Example:

- "Comp Sci"
- "CSE"

API Integration

The chatbot can be deployed as:

- Web application
- Telegram bot
- WhatsApp bot

This would allow students to access the service from multiple platforms.

13. System Implementation

```python
import re

# Sample exam database
exam_schedule = {
    ("CS", "5"): "15 May 2026",
    ("CS", "3"): "10 May 2026",
    ("IT", "5"): "18 May 2026",
    ("IT", "3"): "12 May 2026",
    ("ME", "5"): "20 May 2026"
}

# Entity Extraction Function
def extract_entities(query):

    entities = {
        "semester": None,
        "course": None,
        "date": None
    }

    # Extract Semester
    sem_match = re.search(r'(?:sem|semester)\s*(\d+)', query, re.IGNORECASE)
    if sem_match:
        entities["semester"] = sem_match.group(1)

    # Extract Course / Department
    course_match = re.search(r'\b(CS|IT|ME|CE|EE|EC)\b', query, re.IGNORECASE)
    if course_match:
        entities["course"] = course_match.group(1).upper()

    # Extract Date Words
    date_match = re.search(r'\b(today|tomorrow)\b', query, re.IGNORECASE)
    if date_match:
        entities["date"] = date_match.group(1)

    return entities
```

```python
# Chatbot Loop
def chatbot():

    print("🎓 Exam Query Chatbot")
    print("Type 'exit' to stop.\n")

    while True:

        user_input = input("You: ")

        if user_input.lower() == "exit":
            print("Bot: Goodbye! Best of luck for your exams.")
            break

        entities = extract_entities(user_input)

        response = generate_response(entities)

        print("Bot:", response)


# Run chatbot
chatbot()
```

```
🎓 Exam Query Chatbot
Type 'exit' to stop.

You:  When is the SEM 3 CS exam
Bot: CS Semester 3 exam is scheduled on 10 May 2026.
```

14. Conclusion

The project successfully demonstrates a **basic Natural Language Processing application using rule-based entity recognition**. By leveraging Regular Expressions and dictionary-based storage, the system efficiently extracts key information from user queries and retrieves the corresponding exam schedule.

Although the system relies on predefined rules rather than machine learning models, it offers a **simple, fast, and reliable solution** for structured academic queries where the vocabulary is limited and predictable.

With further enhancements such as database integration, fuzzy matching, and conversational memory, the system can be developed into a more advanced and scalable academic assistant.